Unit 6

Backtracking

Introduction

- Backtracking is a general algorithmic technique for finding solutions to some computational problems, notably constraint satisfaction problems
- Backtracking incrementally builds candidates to the solutions, and abandons a candidate ("backtracks") as soon as it determines that the candidate cannot possibly be completed to a valid solution
- Many of the problems we solve using backtracking require that all solutions satisfy a set of constraints
- Consider a situation that you have three boxes in front of you and only one of them has a gold coin in it but you do not know which one
- So, in order to get the coin, you will have to open all of the boxes one by one
- You will first check the first box, if it does not contain the coin, you will have to close it and check the second box and so on until you find the coin
- This is what backtracking is, that is, solving all sub-problems one by one in order to reach the best possible solution
- Example:
    - There are 5 tunnels and only one (or some) of them lead to an open road on the other side whereas the others lead to a dead end
    - You start from tunnel 1 and explore it
    - If you reach a dead end, you backtrack and then check tunnel 2 and so on
- Example:
    - In crypt arithmetic problems like SEND + MORE = MONEY, you reach a stage like N is either 6 or 7
    - If you check for N = 6 and find out it cannot be the solution, you backtrack and check for N = 7
- Backtracking explores all possible cases/scenarios and produces all outputs that satisfy the given constraints
- Dynamic Programming always only produces the optimal solution

Recursion VS Backtracking

- Recursion is when a function calls itself until a base case is reached
- Backtracking is when an algorithm makes an opportunistic decision, which may come up to be wrong
- If the decision was wrong, then the backtracking algorithm restores the state before the decision
- In backtracking you use recursion in order to explore all the possibilities until you get the best result for the problem

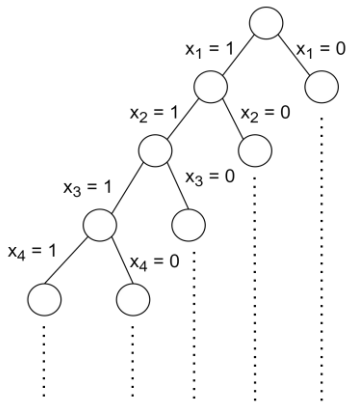
Sum of Subsets

- We are given 'n' distinct positive numbers w_i, $1 \le i \le n$ (usually called weights) and 'm'
- The sum of subsets problem is to find all subsets of w_i whose sums are m
- For example, if n = 4, $(w_1, w_2, w_3, w_4) = (11, 13, 24, 7)$, and m = 31, then the desired subsets are (11, 13, 7) and (24, 7)
- We represent each solution subset by an n-tuple $(x_1, x_2, \ldots, x_n)$ such that $x_i = \{0, 1\}$, $1 \le i \le n$
- Then $x_i = 0$ if w_i is not chosen and $x_i = 1$ if w_i is chosen
- The solutions of the given example are (1, 1, 0, 1) and (0, 0, 1, 1) This formulation expresses all solutions using a fixed-sized tuple
- <u>Eg:</u> n = 6, m = 30, and w[1 : 6] = {5, 10, 12, 13, 15, 18}
  (We assume w_i's are initially in nondecreasing order)
- The solution space consists of 2^n distinct tuples

State Space Tree:

(For a node at

level i, the left child corresponds to $x_i = 1$ and the right to $x_i = 0$)



Bounding Functions

We will apply backtracking and kill nodes if they do not satisfy the bounding functions:

$$B_k(x_1, \ldots, x_k) = true \text{ iff } \sum_{i=1}^{k} w_i x_i + \sum_{i=k+1}^{n} w_i \ge m$$

$$\text{and } \sum_{i=1}^{k} w_i x_i + w_{k+1} \le m$$

Here,

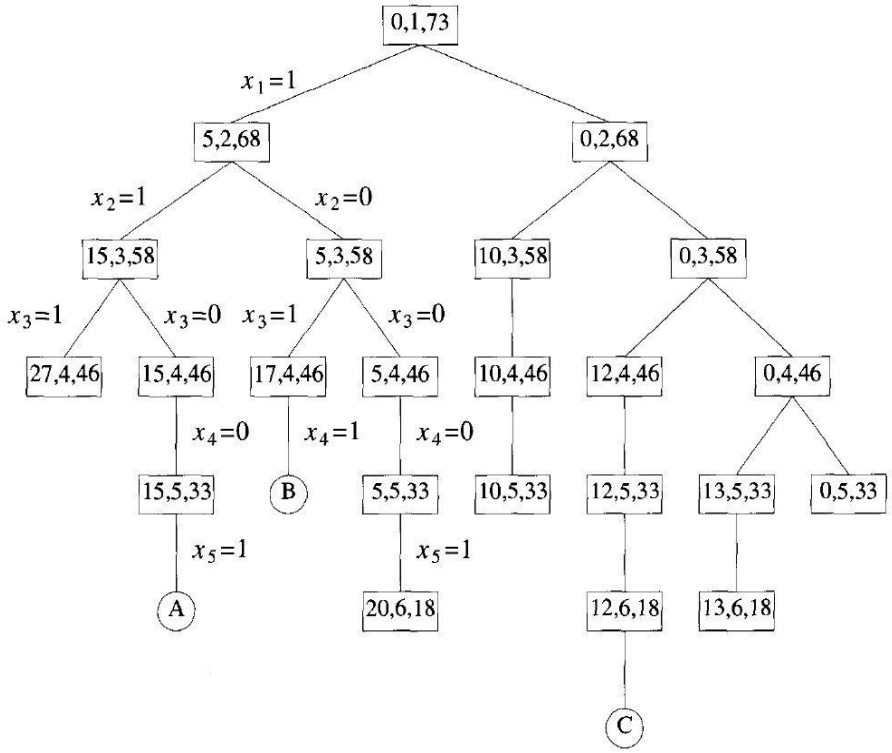
(i = 1 to k) $\sum w_i x_i$ → sum of the weights included so far

(i = k+1 to n) $\sum w_i$ → sum of remaining weights

w_{k+1} → weight of the next object

Sum of Subsets

- The next slide shows the state space tree for the given question with bounding functions applied and when backtracking is used
- The full state space tree for n = 6 contains $2^6 - 1 = 63$ nodes from which calls could be made (this count excludes the 64 leaf nodes as no call need be made from a leaf)



- The given tree shows that we can obtain the sum m = 30 from three subsets indicated by nodes labeled 'A', 'B' and 'C'

  The solutions are:

(1, 1, 0, 0, 1)	[i.e. {5, 10, 15}]
(1, 0, 1, 1)	[i.e. {5, 12, 13}]
(0, 0, 1, 0, 0, 1)	[i.e. {12, 18}]

```
Algorithm SumOfSub(s, k, r)
// Find all subsets of w[1 : n] that sum to m. The values of x[j],
// 1 ≤ j < k, have already been determined. s = Σ_{j=1}^{k-1} w[j] * x[j]
// and r = Σ_{j=k}^{n} w[j]. The w[j]'s are in nondecreasing order.
// It is assumed that w[1] ≤ m and Σ_{i=1}^{n} w[i] ≥ m.
{
    // Generate left child. Note: s + w[k] ≤ m since B_{k-1} is true.
    x[k] := 1;
    if (s + w[k] = m) then write (x[1 : k]); // Subset found
        // There is no recursive call here as w[j] > 0, 1 ≤ j ≤ n.
    else  if (s + w[k] + w[k + 1] ≤ m)
          then SumOfSub(s + w[k], k + 1, r − w[k]);
    // Generate right child and evaluate B_k.
    if ((s + r − w[k] ≥ m) and (s + w[k + 1] ≤ m)) then
    {
        x[k] := 0;
        SumOfSub(s, k + 1, r − w[k]);
    }
}
```

Sum of Subsets

SumOfSub(s, k, r) →	s - sum of weights included so far
	k - checking for weight w_k
	r - sum of remaining weights

The initial call is SumOfSub(0, 1, Σw_i)

The time complexity of Sum of Subsets problem solved using backtracking is $O(2^n)$

0-1 Knapsack Problem

Given n positive weights w_i, n positive profits p_i, and a positive number m that is the knapsack capacity, this problem calls for choosing a subset of weights such that

$$\sum_{1 \le i \le n} w_i x_i \le m \quad \text{and} \quad \sum_{1 \le i \le n} p_i x_i \text{ is maximized}$$

The x_i's constitute a zero-one valued vector

- A good bounding function for this problem is obtained by using an upper bound on the value of the best feasible solution obtainable by expanding the given live node and any of its descendants
- If the upper bound is not higher than the value of the best solution determined so far, then that live node can be killed
- If the items included up to a node have a greater total profit than the best solution so far, we change the value of the best solution so far.
- We always visit a promising node's children
- A node is promising only if we should expand to its children
- If at node Z the values of x_i, $1 \le i \le k$, have already been determined, then an upper bound for Z can be obtained by relaxing the requirement $x_i = 0$ or 1 to $0 \le x_i \le 1$ for $k+1 \le i \le n$ and using the greedy algorithm to solve the relaxed problem
- Function Bound(cp, cw, k) determines an upper bound on the best solution obtainable by expanding any node Z at level k+1 of the state space tree
- The object weights and profits are w[i] and p[i]
- It is assumed that $p[i]/w[i] > p[i+1]/w[i+1]$, $1 \le i \le n$
- We will only use greedy considerations to limit our search; we will not develop a greedy algorithm
- To that end, we first order the items in non-increasing order according to the values of p_i/w_i, where w_i and p_i are the weight and profit, respectively, of the ith item
- Suppose we are trying to determine whether a particular node is promising
- No matter how we choose the remaining items, we cannot obtain a higher profit than we would obtain if we were allowed to use the restrictions in the Fractional Knapsack problem from this node on
- To that end, we first order the items in non-increasing order according to the values of p_i/w_i, where w_i and p_i are the weight and profit, respectively, of the ith item
- Suppose we are trying to determine whether a particular node is promising

- No matter how we choose the remaining items, we cannot obtain a higher profit than we would obtain if we were allowed to use the restrictions in the Fractional Knapsack problem from this node on

```
Algorithm Bound(cp, cw, k)
// cp is the current profit total, cw is the current
// weight total; k is the index of the last removed
// item; and m is the knapsack size.
{
    b := cp; c := cw;
    for i := k + 1 to n do
    {
        c := c + w[i];
        if (c < m) then b := b + p[i];
        else return b + (1 − (c − m)/w[i]) * p[i];
    }
    return b;
}
```

- From Bound it follows that the bound for a feasible left child of a node Z is the same as that for Z
- Hence, the bounding function need not be used whenever the backtracking algorithm makes a move to the left child of a node
- The resulting algorithm is BKnap

```
Algorithm BKnap(k, cp, cw)
// m is the size of the knapsack; n is the number of weights
// and profits. w[ ] and p[ ] are the weights and profits.
// p[i]/w[i] ≥ p[i + 1]/w[i + 1]. fw is the final weight of
// knapsack; fp is the final maximum profit. x[k] = 0 if w[k]
// is not in the knapsack; else x[k] = 1.
{
    // Generate left child.
    if (cw + w[k] ≤ m) then
    {
        y[k] := 1;
        if (k < n) then BKnap(k + 1, cp + p[k], cw + w[k]);
        if ((cp + p[k] > fp) and (k = n)) then
        {
            fp := cp + p[k]; fw := cw + w[k];
            for j := 1 to k do x[j] := y[j];
        }
    }
```

```
19      // Generate right child.
20      if (Bound(cp, cw, k) ≥ fp) then
21      {
22          y[k] := 0; if (k < n) then BKnap(k + 1, cp, cw);
23          if ((cp > fp) and (k = n)) then
24          {
25              fp := cp; fw := cw;
26              for j := 1 to k do x[j] := y[j];
27          }
28      }
29  }
```

Initially fp = -1

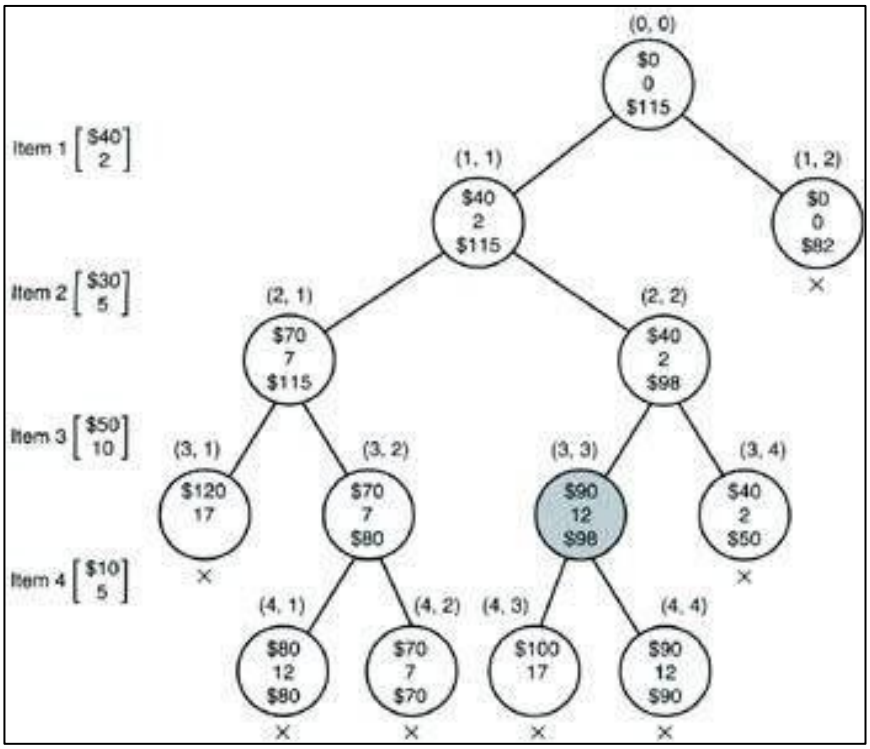
This algorithm is invoked as BKnap(1, 0, 0)

When $fp \neq -1$, $x[i]$, $1 \leq i \leq n$, is such that $\sum_{i=1}^{n} p[i]x[i] = fp$. In lines 8 to 18 left children are generated. In line 20, Bound is used to test whether a right child should be generated. The path $y[i]$, $1 \leq i \leq k$, is the path to the current node. The current weight $cw = \sum_{i=1}^{k-1} w[i]y[i]$ and $cp = \sum_{i=1}^{k-1} p[i]y[i]$. In lines 13 to 17 and 23 to 27 the solution vector is updated if need be.

Example

- The total profit, total weight, and bound are specified from top to bottom at each node
  Suppose that n = 4, m = 16, and we have the following:

i	p_i	w_i	p_i / w_i
1	\$40	2	\$20
2	\$30	5	\$6
3	\$50	10	\$5
4	\$10	5	\$2



Example

- Hence, maximum obtainable profit = $90
- Items included = {1, 3}
- Time complexity of 0-1 Knapsack Problem using backtracking is $O(2^n)$

N-Queens Problem

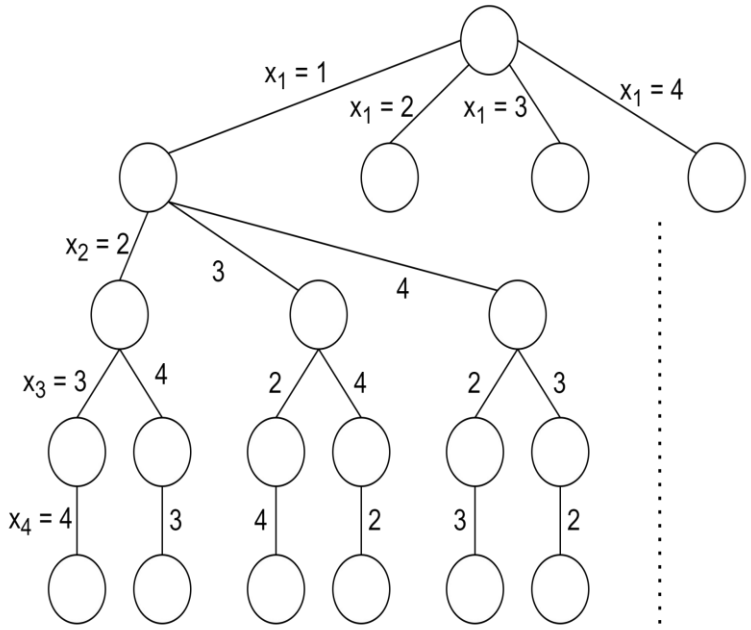
- Place 'n' queens on an n x n chessboard so that no two "attack", i.e., so that no two of them are on the same row, column or diagonal
- Since each queen must be on a different row, without the loss of generality, assume queen 'i' is to be placed on row i
- All solutions to the n-queens problem can be represented as n-tuples (x_1, x_2,.... , x_n), where x_i is the column on which queen i is placed

N-Queens Problem

- The implicit constraints for this problem are that no two x_i's can be the same (i.e., all queens must be on different columns) and no two queens can be on the same diagonal
- 8-queens is the most commonly used n-queens problem
- Consider the n-queens problem for n = 4
- Edges are labeled by possible values of x_i
- Edges from level 1 to level 2 nodes specify the values for x_1 Edges from level i to level (i+1) are labeled with values of x_i

Example

Permutation
Tree:



Example

Total nodes $= 1 + 4 + (4*3) + (4*3*2) + (4*3*2*1)$

$$= 1 + \sum_{i=0}^{3} \left[\prod_{j=0}^{i} (4-j) \right]$$

$= 65$

Example

Total nodes (in n-queens state space tree)

$$= 1 + \sum_{i=0}^{n-1} \left[\prod_{j=0}^{i} (n-j) \right]$$

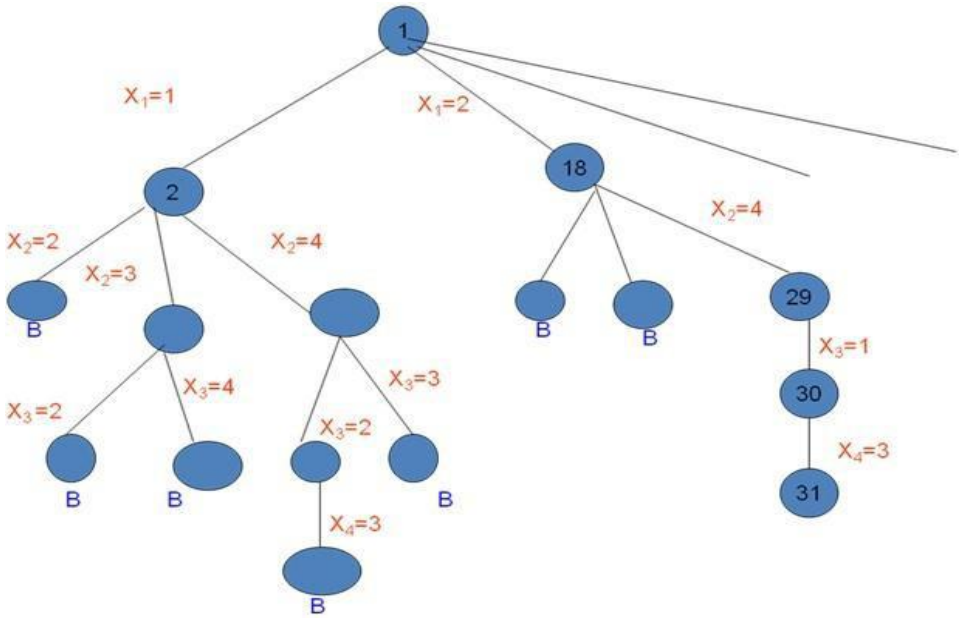
There are $^{n^{\wedge}2}C_n$ possible ways in total (brute force)

eg: For 8x8 chessboard there are $^{64}C_8$ possibilities to place 8 queens

However, by only allowing placement of queens on distinct rows and columns, we require the examination of at most n! n-tuples

Example

Part of the State Space Tree when Bounding Function is used and problem is solved using backtracking



Example

Bounding Function = no same row, column or diagonal for multiple queens

When we complete the tree, we get 2 solutions: (2, 4, 1, 3) and (3, 1, 4, 2)

	Q1		
			Q2
Q3			
		Q4	

Solution 1

		Q1	
Q2			
			Q3
	Q4		

Solution 2

Algorithm

- We can observe that for every element on the same diagonal which runs from the upper left to the lower right, each has the same "(row - column)" value
- Also, every element on the same diagonal which goes from the upper right to the lower left has the same "(row + column)" value
- Suppose two queens are placed at position (k, i) and (j, x[j])
- Then, they are on the same diagonal only if

$$k - i = j - x[j] \quad \text{OR} \quad k + i = j + x[j]$$

From the 1st equation,

$$k - j = i - x[j]$$

From the 2nd equation,

$$i - x[j] = j - k$$

- Therefore, two queens lie on the same diagonal if and only if

| i - x[j] | = | j - k |
OR, | x[j] - i | = | j - k |

Algorithm

Can a new queen be placed?

```
Algorithm Place(k, i)
// Returns true if a queen can be placed in kth row and
// ith column. Otherwise it returns false. x[ ] is a
// global array whose first (k − 1) values have been set.
// Abs(r) returns the absolute value of r.
{
    for j := 1 to k − 1 do
        if ((x[j] = i) // Two in the same column
            or (Abs(x[j] − i) = Abs(j − k)))
                // or in the same diagonal
            then return false;
    return true;
}
```

Prints all solutions to the n-queens problem

```
Algorithm NQueens(k, n)
// Using backtracking, this procedure prints all
// possible placements of n queens on an n × n
// chessboard so that they are nonattacking.
{
    for i := 1 to n do
    {
        if Place(k, i) then
        {
            x[k] := i;
            if (k = n) then write (x[1 : n]);
            else NQueens(k + 1, n);
        }
    }
}
```

Here,

- k = number of the current queen that needs to be placed in the chessboard
- As we know, to place n queens in a nxn chessboard requires checking of n! n-tuples
- Hence, time complexity of the n-queens problem solved using backtracking is O(n!)